

providers, including OpenAI and Hugging Face.

### Use cases.

- Chatbots with memory
- Document summarisation
- Autonomous AI agents

## Rasa

Rasa is an open source framework tailored for building AI-driven chatbots and virtual assistants.

### Key features.

- NLU (natural language understanding) and dialogue management.
- On-premises deployment for data privacy.
- Customisable and extensible architecture.

### Use cases.

- Customer service automation
- Enterprise chatbots
- Voice assistants

Table 3 lists several other projects that contribute to the open source LLM landscape.

## TRAINING AND FINE-TUNING LARGE LANGUAGE MODELS

### Fine-tuning basics

Fine-tuning an LLM involves training a pre-trained model on a smaller, task-specific dataset to improve performance on a particular use case. This is useful when:

- The base model is too general and lacks domain-specific knowledge.
- Performance on a specific task is suboptimal using a pre-trained model.
- Data privacy or customisation is required, and external APIs are not an option.
- Improving efficiency by training a smaller model on targeted data instead of using a massive model.

### Frameworks for fine-tuning

Several frameworks facilitate fine-tuning LLMs efficiently.

**Hugging Face Trainer.** The trainer API from Hugging Face simplifies the training process.

**TABLE 5**  
**BENCHMARK DATASETS**

|           |                                                                                   |
|-----------|-----------------------------------------------------------------------------------|
| GLUE      | A collection of tasks for evaluating natural language understanding (NLU) models. |
| SuperGlue | An advanced version of GLUE designed for more complex language tasks.             |
| ImageNet  | A widely used dataset for image classification models.                            |
| MS COCO   | Used for object detection and image captioning models.                            |
| SQuAD     | A dataset for evaluating question-answering models.                               |

**TABLE 6**  
**DEPLOYMENT TOOLS**

|                            |                                                                                                                                          |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| ONNX                       | Enables interoperability between different deep learning frameworks, allowing models trained in one framework to be deployed in another. |
| TensorRT                   | A high-performance deep learning inference library optimised for Nvidia GPUs.                                                            |
| Hugging Face Inference API | Provides pre-trained models with an easy-to-use API for quick deployment.                                                                |

```
from transformers import Trainer,
TrainingArguments,
AutoModelForSequenceClassification,
AutoTokenizer from datasets import
load_dataset

# Load dataset and model
dataset = load_dataset("imdb")
model = AutoModelForSequenceClassification.
from_pretrained("bert-base-uncased",
num_labels=2)
tokenizer = AutoTokenizer.from_
pretrained("bert-base-uncased")

def preprocess_data(examples):
    return tokenizer(examples["text"],
truncation=True, padding=True)

dataset = dataset.map(preprocess_data,
batched=True)

# Define training arguments
training_args = TrainingArguments(
    output_dir="./results", per_device_
train_batch_size=8, num_train_epochs=3,
logging_dir="./logs"
)

# Trainer instance
trainer = Trainer(
    model=model, args=training_args,
train_dataset=dataset["train"], eval_
dataset=dataset["test"]
)

trainer.train()
```

**PyTorch Lightning.** PyTorch

Lightning offers a structured approach for scalable model training.

```
import pytorch_lightning as pl
from transformers import AutoModel,
AutoTokenizer class
LLMFineTuner(pl.LightningModule):
    def __init__(self, model_name="
bert-base-uncased", num_labels=2):
        super().__init__()
        self.model = AutoModel.from_
pretrained(model_name, num_labels=num_
labels)

    def forward(self, x):
        return self.model(x)
```

### Distributed training

For large scale fine-tuning, distributed training tools help efficiently utilise multiple GPUs or TPUs.

**DeepSpeed.** Optimises large scale training with memory-efficient techniques.

**Horovod.** Open source framework for distributed deep learning.

**Ray Train.** Provides scalable model training across clusters.

Here's an example for enabling DeepSpeed in Hugging Face Trainer:

```
training_args = TrainingArguments(
    output_dir="./results",
    per_device_train_batch_size=8,
    num_train_epochs=3,
    logging_dir="./logs",
    deepspeed="ds_config.json" #
DeepSpeed config file
)
```